WEEK 06

Classes, instances and methods

Research 4 exchange · Class warm-up before Session 1

Python foundations

Classes, instances and methods

Classes, instances and returned values

Class and instance

A class defines a type; a call constructs an instance.

```
class Counter: pass
first = Counter(); second = Counter()
```

Attributes belong to objects

Separate instances can store different values.

```
class Counter: pass
first = Counter(); first.count = 4
```

Methods receive self

A method receives the object whose data it uses.

```
class Counter:
    def doubled(self): return self.count * 2
```

One class, two separate objects

Rectangle class

width · height · area()

first



width 3 · height 4

`area() → 12`

second



width 2 · height 5

`area() → 10`

```
first.width = 1
print(first.area(), second.area())  # 4 10
```

The method definition is shared; self identifies the instance used in this call.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
class Name:	Define a type	Group related data and behavior
self.value	One instance attribute	Read or change this object
obj.method()	Call a method	Use the receiving instance
alias = obj	Share an object	Both names refer to one instance

Python Basics · sections 18, 13, 16 · use only when you need a lookup.

Construct one object and read attributes

```
01 class Label:
02     def __init__(self, text):
03         self.text = text
04
05 label = Label("A")
06 print(label.text)
```

1. class defines a type; calling it creates one instance.

Call a method and use its returned value

```
01 class Counter:
02     def __init__(self, value):
03         self.value = value
04
05     def next_value(self):
06         return self.value + 1
07
08 counter = Counter(3)
09 print(counter.next_value())
10 print(counter.value)
```

1. A method is a function on an object.

Separate instances have separate attributes

```
01 class Counter:
02     def __init__(self, value):
03         self.value = value
04
05 first = Counter(1)
06 second = Counter(2)
07 first.value = 9
08 print(first.value, second.value)
```

1. Calling the class twice creates two objects.

Two objects share a class, not their attributes

```
01 class Counter:
02     def __init__(self, count):
03         self.count = count
04     def doubled(self):
05         return 2 * self.count
06 first = Counter(4)
07 second = Counter(2)
08 print(first.doubled(), second.doubled())
```

1. Expected: 8 4.
2. Trace which instance is `self` on each call.
3. Change first.count; inspect both objects.

Hands-on practice

Week_06.ipynb · Session 2

Python practice

P1–P2 · Core

Two small programs with approach hints.

P3 · Core

A complete program with normal and edge-case tests.

P4–P5 · Optional

An application and a harder challenge.

P1–P3: your code, expected results and an explanation. P4–P5 are optional.

Project studio

Persistent plans and line snapshots

The project need

1

Need

The next volunteer needs to find the plan tomorrow.

2

Operation

Save a plan and retrieve it after closing and reopening the database.

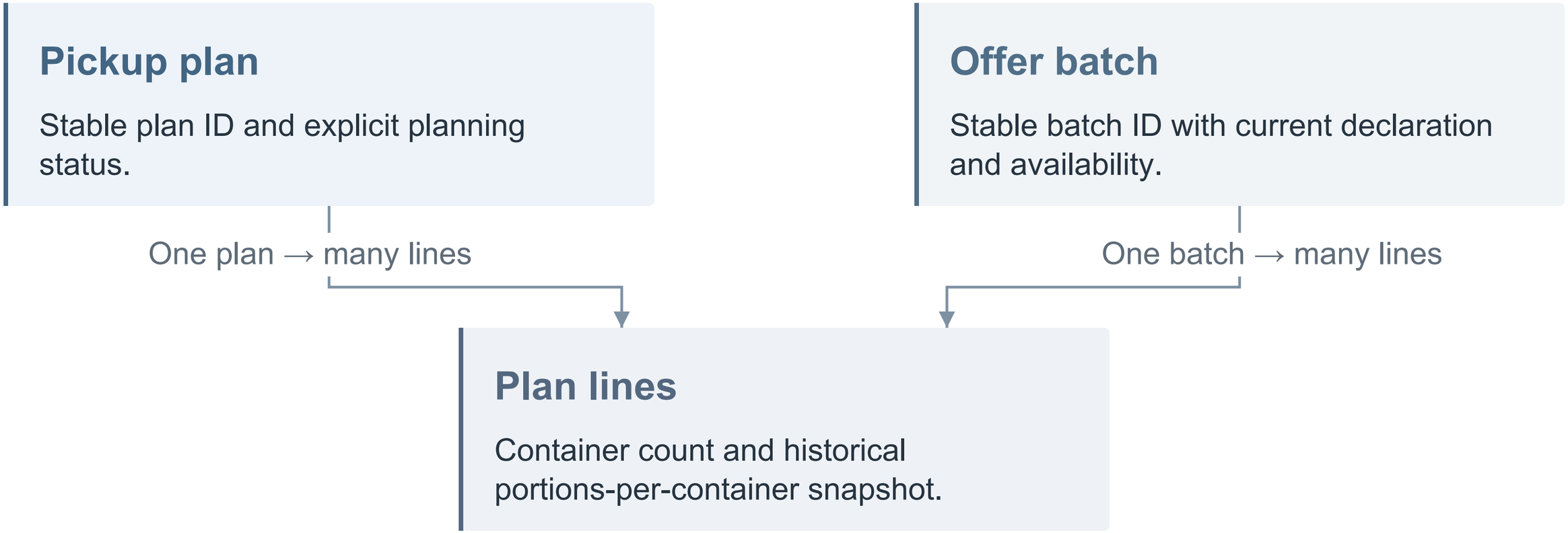
3

Result

The next volunteer can find the same saved facts.

Planning is not saving: the supplied engine handles stored records.

Connect a plan to its batch-line records



One plan contains lines; one batch may appear in many plans.

Reference: Define a class with attributes and a method

```
class OfferLine:
    def __init__(self, batch_id,
                 portions_per_container, containers):
        self.batch_id = batch_id
        self.portions_per_container = (
            portions_per_container)
        self.containers = containers

    def total_portions(self):
        return (self.portions_per_container
                * self.containers)
```

1. __init__ sets the instance attributes.
2. self refers to the current instance.
3. The method returns a calculated value.

A class groups instance data and related methods.

Reference: Two instances hold separate values

```
from rescue_models import OfferLine

first = OfferLine("DEMO", 3, 4)
second = OfferLine("DEMO", 3, 2)
print(first.total_portions(), second.total_portions())
assert first.total_portions() == 12
assert second.total_portions() == 6
```

1. Expected planning totals: twelve and six
2. An alias is not a new independent object
3. Public mutation bypasses constructor checks

Python supplies the receiving instance as self.

SQL parameters and a returned row

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE plans(id TEXT, portions INT)")
db.execute("INSERT INTO plans VALUES (?, ?)",
           ("DEMO-P", 12))
row = db.execute("SELECT portions FROM plans WHERE id=?",
                 ("DEMO-P",)).fetchone()
assert row == (12,)
print(row)
db.close()
```

1. Pass values separately from SQL text
2. The one-element tuple needs its comma
3. This memory-only example is not persistence evidence

A one-column result is a tuple; no match is None.

Paths: current directory and resolved filename

```
01 from pathlib import Path
02
03 print("Working directory:", Path.cwd())
04 path = Path("pickup_plans.sqlite").resolve()
05 print("Would open:", path)
06 print("Exists:", path.exists())
```

1. This inspection creates no database
2. A new SQLite path may create an empty file
3. Never reset useful records just to diagnose a path

A relative filename can refer to a different file from another directory.

File persistence: commit, close and reopen

```
from tempfile import TemporaryDirectory
from pathlib import Path
import sqlite3
with TemporaryDirectory() as folder:
    path = Path(folder) / "plans.sqlite"
    db = sqlite3.connect(path)
    db.execute("CREATE TABLE plans(portions INT)")
    with db: db.execute("INSERT INTO plans VALUES (12)")
    db.close()
    db = sqlite3.connect(path)
    print(db.execute("SELECT * FROM plans").fetchone())
    db.close()
```

1. Expected stored result: (12,)
2. Only the newly created practice folder is removed
3. The notebook also uses a fresh reader process

The second connection reads committed file state.

None: handle a missing query result

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE plans(id TEXT, portions INT)")
row = db.execute("SELECT portions FROM plans WHERE id=?",
                 ("unknown",)).fetchone()
if row is None:
    print("Plan not found")
else:
    print(row[0])
db.close()
```

1. A missing row returns None.
2. Use is None to choose the branch.
3. Only index the row when one exists.

Check for None before reading a selected column.

Project task

Supplied starting point

Schema, fixture and SQL examples are supplied. Python objects are a modelling example, not a required rewrite.

You run / explain

Save P11 with Q11: 2 × 3 and Q12: 1 × 5. Commit, close, reconnect and retrieve its header and lines; handle a missing lookup.

Check the evidence

Saved total stays 11 after Q11 unit changes 3 → 4; current recalculation becomes 13. Availability stays Q11=10, Q12=4.

Recover the same planning estimate without changing batch availability.

Project checks

Step	What to show
Implementation	Save P11 and line snapshots; reopen and retrieve.
Checks	Saved 11; current recalculation 13; stock stays 10/4.

Recover the same planning estimate without changing batch availability.

Further reading in English

1. ## Python classes and instance methods

   https://docs.python.org/3/tutorial/classes.html

2. ## Python sqlite3 documentation

   https://docs.python.org/3/library/sqlite3.html

Read one relevant section, then bring an example and a question.